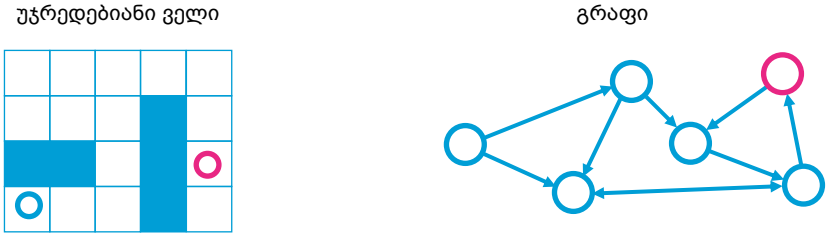
6.4 გზის ძიების ალგორითმები
Pathfinding algorithms

Viam supervadet vadens

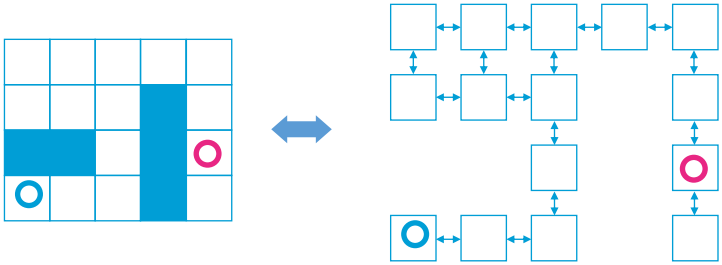
ამ თავში განვიხილავთ რამდენიმე ალგორითმს, რომლებიც პოულობენ გზას მოცემულ ამოცანაში.

გზის ძიების ალგორითმების გამოყენების სფერო დიდია, კერძოდ, ის გამოიყენება ელექტრონიკაში, საბეჭდი პლატების დაპროექტებისთვის. ფართო მომხმარებლისთვის უფრო თვალსაჩინო მაგალითია კომპიუტერულ თამაშებში პერსონაჟების გადაადგილების მარშრუტების გამოთვლა, ან თუნდაც რეალურ სამყაროში ნავიგაციის დაგეგმარება.

მაგალითები იქნება განხილულო ორი ტიპის მონაცემებზე: უჯრედებიან ველზე და გრაფზე (სურ.). ორი მაგალითი პირველ ტიპზე და ერთი - მეორეზე. თუმცა ალგორითმების იდეა უნიოვერსალურია და ეს ალგორითმები, შესაბამისი მოდიფიცირებით, ორივე ტიპზე მუშაობს

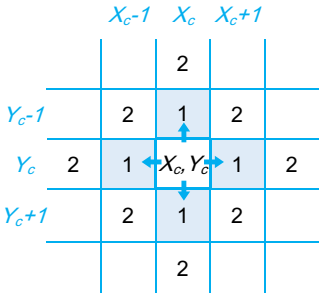


სინამდვილეში უჯრებიანი ველიც შეგვიძლია წარმოვიდგინოთ როგორც გრაფი, თანაც რამდენიმე მეთოდით. სურათზე მოყვანილია ყველაზე მარტივი გზა, როდესაც ველის ყოველი უჯრედი გრაფის კვანძს წარმოადგენს



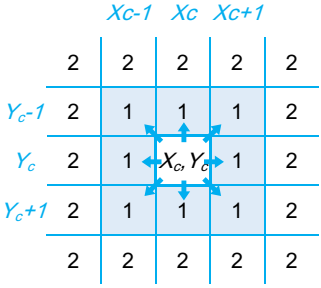
სიმარტივისთვის რამდენიმე მაგალითს გავივლით უჯრედიანი ველის მაგალითზე, ხოლო თვითონ ველი იქნება ფორმალიზეული არა როგორც გრაფი, არამედ როგორც ორგანზომილებიანი მასივი. სანამ დავიწყებთ ამ ალგორითმების გავლას, უნდა მოვილაპარაკოთ იმაზე, თუ რა არის უჯრედის სამეზობლო.

ფონ ნოიმანის და მურის სამეზობლოები

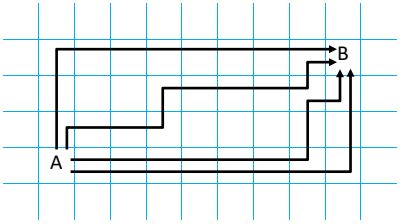


ფონ ნოიმანის სამეზობლო (**Von Neumann neighborhood**): ყოველი უჯრედისთვის, რომლის კოორდინატებია (X_c, Y_c), მეზობელ უჯრედებად ითვლება უჯრედები კოორდინატებით: (X_c-1, Y_c), (X_c, Y_c-1), (X_c+1, Y_c), (X_c, Y_c+1) (სურ.) , ანუ მიმდინარე უჯრედიდან დაშორებული ერთი ვერტიკალური ან ჰორიზონტალური გადასვლით.

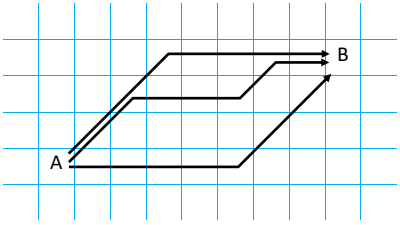
1 - საწყისი უჯრედიდან ერთი ნაბიჯის მანძილზე მდებარე უჯრედები
2 - საწყისი უჯრედიდან ორი ნაბიჯის მანძილზე მდებარე ელემენტები



მურის სამეზობლო (**Moore neighborhood**): ყოველი უჯრედისთვის, რომლის კოორდინატებია (X_c, Y_c), მეზობელ უჯრედებად ითვლება უჯრედები, დაშორებული ერთი ვერტიკალური, ჰორიზონტალური ან დიაგონალური გადასვლით. ეს ემთხვევა ჭადრაკში მეფის ფიგურის სვლებს.

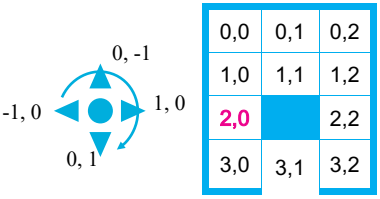


მანჰეტენის დისტანცია (Manhattan distance): თუ გადაადგილება შესაძლებელია მხოლოდ ფონ ნოიმანის სამეზობლოში, მაშინ ორ წერტილს შორის მანძილი მათი კოორდინატების სხვაობის ჯამის ტოლია: $|B_x-A_x| + |B_y-A_y|$. მოცემულ მაგალითში ყველა გზა გვაძლევს ერთ და იგივე დისტანციას D = 11.

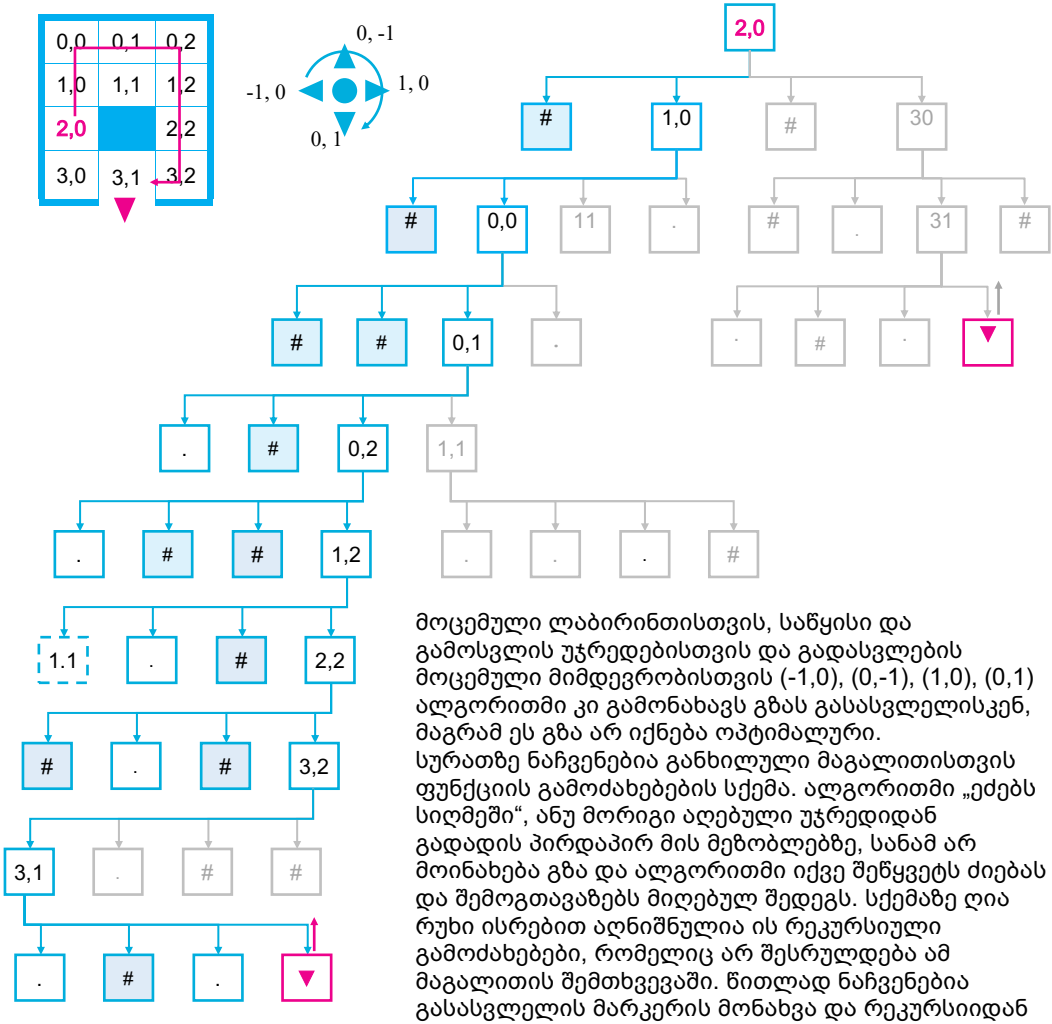


თუ კი გადასვლები მურის სამეზობლოში ხდება, დისტანცია მცირდება: შესაძლოა დიაგონალური სვლა: ამ შემთხვევაში D = 8.

ახლა კი დავუბრუნდეთ ლაბირინთიდან გამოსვლის რეკურსიულ ალგორითმს, რომელიც განვიხილეთ თავში XXXX.
ალგორითმი იძახებდა თავის თავს რეკურსიულად - ოთხჯერ ყოველი მიმდინარე კვანძისთვის (ანუ ვიყენებდით ფონ ნოიმნის სქემას). ასეთი ხისებრი რეკურსიულო გამოძახება წარმოადგენს ძიებას სიღრმეში (DFS - Depth First Search) და ის გვიბრუნებს პირველ მოძიებულ გზას, რაც სულ არ ნიშნავს ამოცანის ოპტიმალურ ამოხსნას.



სურათზე მოყვანილია მარტივი ველი, სადაც უნდა ვნახოთ გზა წერტილ 20-იდან ლაბირინთის გასასვლელამდე. თუ ავიღებთ იგივე გადასვლების მიმდევრობას, რაც აღწერილი იყო xxxx თავში (მარცხნივ, მაღლა, მარჯვნივ, დაბლა - სურ. xxxx), მაშინ გვექნება ფუნქციის გამოძახებების შემდეგი ხე:



მოცემული ლაბირინთისთვის, საწყისი და გამოსვლის უჯრედებისთვის და გადასვლების მოცემული მიმდევრობისთვის (-1,0), (0,-1), (1,0), (0,1) ალგორითმი კი გამონახავს გზას გასასვლელისკენ, მაგრამ ეს გზა არ იქნება ოპტიმალური.
სურათზე ნაჩვენებია განხილული მაგალითისთვის ფუნქციის გამოძახებების სქემა. ალგორითმი „ეძებს სიღმეში“, ანუ მორიგი აღებული უჯრედიდან გადადის პირდაპირ მის მეზობლებზე, სანამ არ მოინახება გზა და ალგორითმი იქვე შეწყვეტს ძიებას და შემოგთავაზებს მიღებულ შედეგს. სქემაზე ღია რუხი ისრებით აღნიშნულია ის რეკურსიული გამოძახებები, რომელიც არ შესრულდება ამ მაგალითის შემთხვევაში. წითლად ნაჩვენებია გასასვლელის მარკერის მონახვა და რეკურსიიდან გამოსვლა.

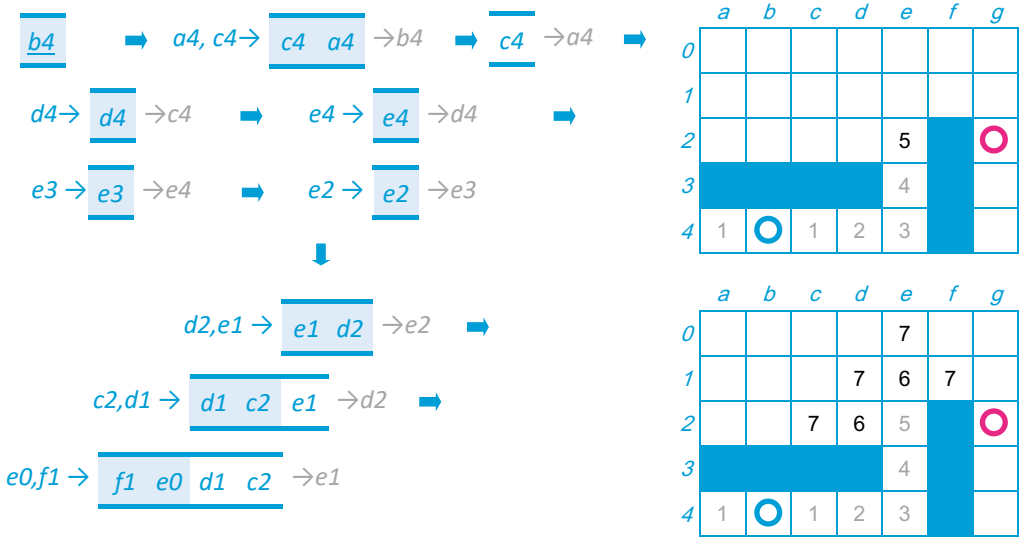
ამავდროულად ჩვენ ვხედავთ, რომ გამოძახებების ხის მარჯვენა, შეუსრულებელ, „რუხ“ ნაწილში არის გაცილებით უფრო მოკლე გზა: 20, 30, 31.
რომ შეგვეცვალა პროგრამულ კოდში გადასვლების მიმდევრობა ისე, რომ უჯრედიდან ჯერ ყოფილიყო გადასვლა ჯერ ქვემოდ, მერე კი მარჯვნივ, მაშინ თავიდანვე მოიძებნებოდა ეს მოკლე გზა. (არ დაგვავიწყდეს, რომ არჩეულია კოორდინატთა სისტემა, სადაც სურათზე „ქვემოთ“ ნიშნავს Y-ს გაზრდას).
მაგრამ ალგორითმის კოდირების დროს ჩვენ არ ვიცით მომავალი ლაბირინთების კონფიგურაცია და ძალზედ სასურველია, რომ ალგორითმის მუშაობის ხარისხი არ იყოს დამოკიდებული შემთხვევაზე.

ზემოდმოყვანილი გამოძახებების ხე რომ წაგვეკითხა „სტრიქონებად“, ანუ BFS (Breadth -First Search), მაშინ, ამ ხის ზემოდან ქვემოდ გავლისას, პირველი შეგვხდებოდა უმოკლესი გზა.
ასე რომ, გადავდივართ სიგანეში ძიებაზე. ამისთვის დროებით დავემშვიდობოთ რეკურსიას და გავიხსენოთ რიგის (queue) გამოყენება.

ლის ალგორითმი (the Lee algorithm)

ვიწყებთ საწყისი უჯრედიდან, რომელსაც თავიდან ვამატებთ რიგში (queue). ვანიჭებთ ამ უჯრედს მნიშვნელობას 0.
შემდეგ გადავდივართ ყველა მის დასაშვებ მეზობელ უჯრედზე (ანუ არა კედლის უჯრედზე და არა უკვე გავლილ უჯრედზე) და მათ ვანიჭებთ მნიშვნელობა 1-ს. ამ უჯრედებს ვამატებთ რიგში, ხოლო საწყის უჯრედს ამ რიგიდან ვშლით.
რიგში ჩაწერილ უჯრედს თანმიმდევრობით ვამუშავებთ იგივე მეთოდით, ოღონდ ყოველი უჯრედის მეზობლებს მასზე ერთით დიდი მნიშვნელობა ენიჭება.
ციკლს ვაგრძელებთ მანამ, სანამ რიგი არ დაცარიელდება ან არ შეგვხდება მიზნობრივი უჯრედი (თუ ლაბირინთია - გასასვლელი, ხოლო თუ ორ წერტილს შორის გზას ვგეგმავთ ის წერტილი, რომელშიც უნდა მივიდეთ)

იმ კვანძს, რომელსაც ვშლით რიგიდან, „მოყვას“ რიგში თავისი მეზობლები (თუ ისინი უკვე არ არიან იქ) და ანიჭებს მათ თავის თავზე ერთით დიდ მნიშვნელობას:



აქ კარგად ჩანს, რომ კვანძებზე მნიშვნელობები ვრცელდება როგორც ტალღა, ერთი მნიშვნელობის მქონე კვანძები ტალღის ფრონტივით არის განლაგებული

a2, b1 → b1 a2 g1 f0 d0 c1 →b2 ➡

c0 → c0 b1 a2 g1 f0 d0 →c1 ➡

c0 b1 a2 g1 f0 →d0 ➡

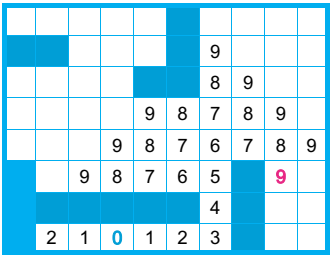
g0 → g0 c0 b1 a2 g1 →f0 ➡ g2 → g2 g0 c0 b1 a2 →g1

	a	b	c	d	e	f	g
0			9	8	7	8	9
1		9	8	7	6	7	8
2	9	8	7	6	5		9
3					4		
4	1		1	2	3		

ასე მოვედით g2 კვანძამდე, რომელსაც მიენიჭა მნიშვნელობა 9, რაც გავლილი გზის სიგრძის ტოლია. იმისთვის, რომ აღვადგინოთ გავლილი გზა, საკმარისია, ბოლო კვანძიდან (მოცემულ შემთხვევაში - g2) დაწყებული, ავიღოთ ის მეზობელი, რომლის მნიშვნელობა მიმდინარე კვანძის მნიშვნელობაზე ერთით ნაკლებია, გადავიდეთ იმ კვანძზე და ასე გავაგრძელოთ, სანამ არ მივაღწევთ საწყის კვანძს:

	a	b	c	d	e	f	g
0			9	8	7	8	9
1		9	8	7	6	7	8
2	9	8	7	6	5		9
3					4		
4	1		1	2	3		

მოცემული მაგალითიდან ჩანს, რომ რიგი კვანძი საერთოდ არ იყო მონახული. ზოგადად, რაც უფრო დიდია ველი გავლილ გზასთან შედარებით და რაც უფრო „თავისუფალია“ ის (ანუ ბევრია თავისუფალი სივრცე და შედარებით ნაკლებია ვიწრო „კორიდორები“) მით უფრო ეფექტურია ეს ალგორითმი სიღრმეში ძიებასთან შედარებით. სურათზე ნაჩვენებია zoom out: ალგორითმის მიერ გავლილი ზონა უფრო დიდი ველის შემთხვევაში .



აქვე აღვნიშნავთ, რომ ეს ალგრითმი შეგვიძლია გამოვიყენოთ ასევე კომპ. გრაფიკაში ფერის ჩასხმის ამოცანისთვის (color fill)

ალგორითმი ფსევდოკოდში:
ველის ყველა უჯრედს ენიჭება მნიშვნელობა -1 („ჯერ არ დამუშავებული“)
საწყისი უჯრედის კოორდინატები შეგვყავს რიგში (queue)
ამ საწყისი უჯრედის მნიშვნელობა ხდება 0
ციკლი (სანამ რიგი არ არის ცარიელი):
    რიგიდან ამოიღება მორიგი უჯრედის კოორდინატები
    ამ უჯრედის ყველა მეზობლისთვის, ვისი მნიშვნელობაც არის (–1):
        თუ ეს მეზობელი - სამიზნე უჯრედია, გავდივართ ციკლიდან;
        მეზობელი ღებულობს მორიგი უჯრედის მნიშვნელობას +1,
        მეზობელი ემატება რიგში
ციკლის დასასრული

ვიღებთ სამიზნე უჯრედის კოორდინატებს.
თუ სამიზნე უჯრედის მნიშვნელობა უდრის –1-ს, გზა არ არსებობს
ციკლი (სანამ უჯრედი არ არის საწყისი უჯრედი):
    ახალი უჯრედი - ის, რომლის მიმდინარეზე ერთით ნაკლებია
ციკლის დასასრული

A* (A-star) ალგორითმი
A* (“A-star”) algorithm

2.3

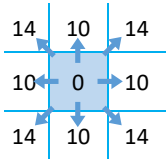
Two roads diverged in a wood, and I - I took the one less traveled by

გზის დაგეგმარების სხვა მეთოდი - ე.წ. A* (“A-star”, “A* Pathfinding”) ალგორითმი.

სანამ აღვწერთ თვითონ ალგორითმს, განვიხილოთ ამოცანის გეომეტრია, გადაადგილების წესები და კვანძებს შორის დისტანციის გაანგარიშების მეთოდი

სიმარტივისთვის ავიღოთ კვადრატებად დაყოფილი სიბრტყე. ქვემოთ განხილულ მაგალითში გადაადგილება შეიძლება როგორც ვერტიკალური და ჰორიზონტალური მიმართულებებით (დავარქვათ ამას ორთოგონალური მოძრაობა), ასევე დიაგონალური მიმართულებებით (ანუ მურის სამეზობლო).
იმის გამო, რომ დიაგონალური გზა sqrt(2) -ჯერ უფრო გრძელია, ვიდრე ორთოგონალური, დიაგონალური გადაადგილება შევაფასოთ 1,4-ჯერ უფრო ძვირად.

იმისთვის, რომ ალგორითმმა იმუშავოს უფრო სწრაფად, გამოვიყენოთ მთელი რიცხვების არითმეტიკა: ორთოგონალური და დიაგონალური გადასვლების ფასები/მანძილები ავიღოთ არა 1 და 1.4, არამეტ 10 და 14. (სურ.)



მანძილი კვანძებს შორის იზომება სვლებში, ანუ გავლილი კვანძების რაოდენობაში. სურათზე ნაჩვენებია ორ კვანძს შორის რამდენიმე უმოკლესი გზიდან სამი. სამივე შემთხვევაში გვაქვს ორი ორთოგონალური და სამი დიაგონალური სხვა, ანუ 2*10+3*14 = 62. სულ ხუთი სვლა. ამავდროულად, რომ წავსულვიყავით მხოლოდ ვერტიკალური ან ჰორიზონტალური გზით (ფონ ნოიმანის სამეზობლო), გვექნებოდა 8 სვლა სიგრძით 80.

24	20	24	28	38	48	58	68
14	10	14	24	34	44	54	64
10	0	10	20	30	40	50	60
14	10	14	24	34	44	54	64
24	20	24	28	38	48	58	68
34	30	34	38	42	52	62	72
44	40	44	48	52	56	66	76

მოცემული კვანძიდან ნებისმიერ სხვა კვანძამდე მურის დისტანციის გაანგარიშება მოხდება შემდეგნაირად: უნდა ავიღოთ კვანძების კოორდინატების სხვაობები: DX და DY. მათ შორის უფრო პატარა რიცხვი იქნება დიაგონალური გადასვლების რაოდენობა, ხოლო ამ რიცხვების განსხვავება - ორთოგონალური სვლების რაოდენობა:

ჩვენ მაგალითში DX = 5, DY = 3. შესაბამისად:
min(DX, DY) = 3 - ამდენი დიაგონალური სვლაა, abs(DX-DY) = 2 - ამდენი ორთოგონალური სვლაა. შესაბამისად, მოცემული კვანძიდან კოორდინატებით ix, iy კვანძამდე კოორდინატებით jx, jy მანძილი იქნება:

```
dx = abs(ix-jx);  dy = abs(iy-jy);
L = min(dx, dy) * 14 + abs(dx-dy) * 10;
```

სურათზე კვანძების მეორე მხარეს ჩავწეროთ მანძილები „სამიზნე“ კვანძამდე (ლურჯი რიცხვები). დავარქვათ მათ Hcost. თუ ყოველი კვანძისთვის შევაჯამებთ მის „შავ“ და „ლურჯ“ რიცხვებს, მივიღებთ ჯამურ მანძილს ორივე (საწყის და ბოლო) კვანძამდე. („წითელი რიცხვები“):

Fcost = Gcost + Hcost

და მართლაც: ყოველი უმოკლესი გზა გადის იმ კვანძებზე, რომლებიც რაც შეიძლება ახლოსაა როგორც საწყის, ასევე ბოლო კვანძთან.

24	80	20	70	24	66	28	62	38	58	48	54	58	50	68	54
104		90		90		90		96		102		108		122	
14	76	10	66	14	56	24	52	34	48	44	44	54	40	64	44
90		76		70		76		82		88		94		108	
10	72	0	62	10	52	20	42	30	38	40	34	50	30	60	34
82		62		62		62		68		74		80		94	
14	68	10	58	14	48	24	38	34	28	44	24	54	20	64	24
82		68		62		62		62		68		74		88	
24	64	20	54	24	44	28	34	38	24	48	14	58	10	68	14
88		74		68		62		62		62		68		82	
34	60	30	50	34	40	38	30	42	20	52	10	62	0	72	10
94		80		74		68		62		62		62		82	
44	64	40	54	44	44	48	34	52	24	56	14	66	10	76	14
108		94		88		82		76		70		76		90	

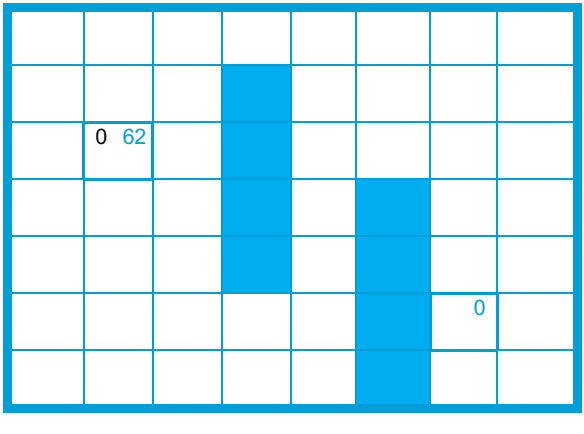
სურათზე ლურჯად არის აღნიშნული ველები ყველაზე დაბალი ჯამური დისტანციით (62). ყველა 6-ნაბიჯიანი გზა საწყისი წერტილიდან ბოლო წერტილამდე იქნება ამ ზონაში.

ამ შემთხვევაში შეგვიძლია ალგორითმი შემდეგნაირად აღვწეროთ:

- დაწყებული საწყისი კვანძით, გადავდივართ კვანძზე, რომლის ჯამური მანძილი ყველაზე მცირეა.
- თუ ასეთი კვანძი რამდენიმეა, მაშინ გადავდივართ იმ კვანძზე, რომლის მანძილი ბოლო წერტილამდე („ლურჯი მანძილი“, Hcost) უფრო დაბალია.
- ახალ კვანძზე გადასსვლისას ვიმახსოვრებთ მის წინამორბედს.
- თუ მანძლი ბოლო კვანძამდე გახდა ნოლი, ვამთავრებთ სვლას.

„Гладко было на бумаге да забыли про овраги”: და რა ვქნათ, თუ გზაზე დაბრკოლებებია? თან ისეთი, რომ მთლიანად გამორიცხავს ოპტიმალური გზის ყველა ვარიანტს? (სურ).

ალგორითმის ძირითადი იდეა არ იცვლება - ვეძებთ კვანძს, სადაც ორივე მანძილის (საწყის წერტილამდე და ბოლო წერტილამდე) ჯამი მინიმალურია. საწყის წერტილამდე მანძილი გამოითვლება თანდათანობით, ხოლო ბოლო კვანძამდე მანძილი შეგვიძლია ავიღოთ, კედლების გათვალისწინების გარეშე - იგივე მეთოდით, რაც წინა მაგალითში - მხოლოდ კოორდინატების გათვალისწინებით (სურ.)



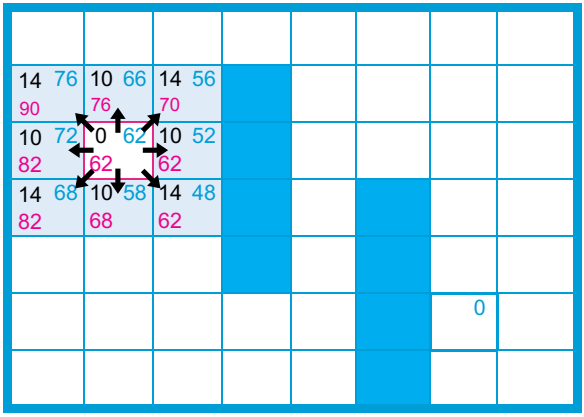
აქ გამოვიყენებთ ე.წ. ევრისტიკას.

ევრისტიკა ინფორმატიკაში არის გარკვეული წესი, რომლის დაცვა ზოგადად გვეხმარება ამოცანის ამოხსნისთვის. ამ წესმა შეიძლება რომელიმე კონკრეტულ შემთხვევაში ვერ იმუშავოს, მაგრამ უფრო ხშირად ის გვეხმარება სწორი გზის პოვნაში

ჩვენ შემთხვევაში ევრისტიკა იქნება: „მიმდინარე კვანძიდან გადავდივართ იმ მეზობელზე, რომლის მანძილი ბოლო წერტილამდე მინიმალურია“.

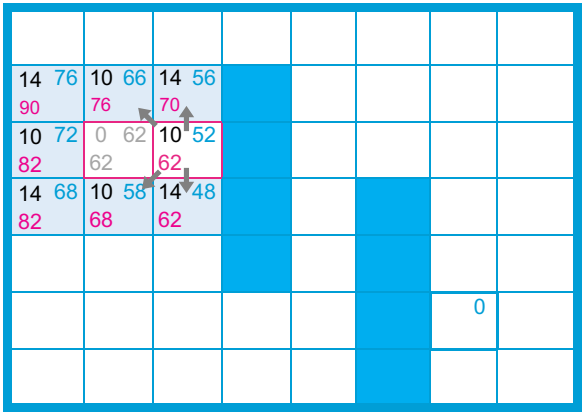
წინა მაგალითში ეს არ იყო ევრისტიკა, რათგან გარანტირებულად გვაძლევდა ოპტიმალურ შედეგს. ახლა კი ეს წესი კარგავს თავის “აბსოლუტურობას” და გადადის „რეკომენდაციების“ კატეგორიაში: ზოგ შემთხვევაში ევრისტიკა ვერ იმუშავებს და სხვა გზის ძებნა დაგვჭირდება.

ვიწყებთ საწყისი კვანძიდან. სქემაზე ღია ლურჯი ფერით აღნიშნულია ის კვანძები, რაც მას ესაზღვრება. თვითონ საწყისი წერტილი „იხურება“, ანუ აღინიშნება როგორც გავლილი (სურათზე - წითელი ჩარჩო)



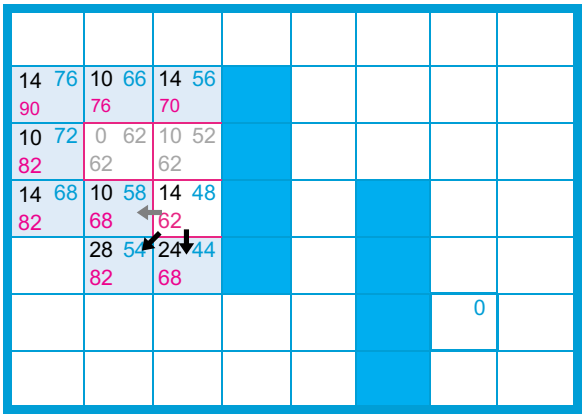
პირველი გადასვლით ევრისტიკა გვთავაზობს ორ კვანძს, ორივე ჯამური მნიშვნელობით 62. ამ ორიდან ჯერ განიხილება კვანძი მინიმალური Gcost - ით (10).

ამ კვანძზე გადასვლისას ახალ კვანძებს ვერ ვხსნით: მარჯვნივ— კედელია, მარცხნივ - დახურული კვანძი. დანარჩენი ოთხივე მოსაზღვრო კვანძი უკვე გახსნილია. მათზე გადასვლას აზრი ექნებოდა მხოლოდ იმ შემთხვევაში, თუ ეს მათ Gcost-ს შეამცირებდა, მაგრამ მოცემულ მაგალითში ეს ვერ მოხდება: მიმდონარე წერტილიდან გადასვლა მოგვცემდა Gcost მნიშვნელობებს 10+14 ან 10+14, ხოლო ამ მეზობელი კვანძების მნიშვნელობებია 10 და 14, რაც ნაკლებია.



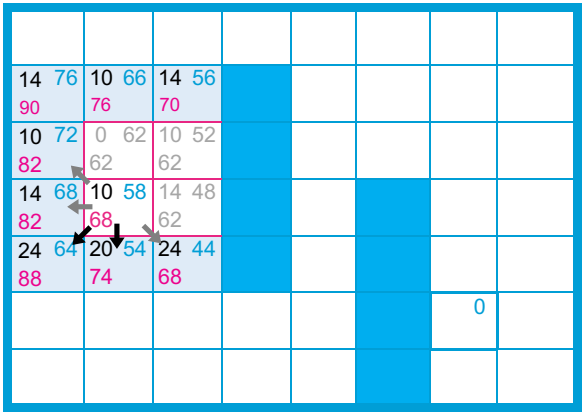
ახლა გახსნილ კვანძებში ვეძებთ შემდეგ გვანძს მინიმალური Fcost - ით. ეს იქნება კვანძი 62 (14,48).

ამ კვანძზე გადასვლა გვიხსნის ორ ახალ კვანძს (სურ.)



აქვე აღსანიშნავია, რომ ერთერთ მათგანს, კვანძს 82(28,54) Gcost უფრო მაღალი მნიშვნელობა მიენიჭა (28), ვიდრე უნდა ჰქონოდა (20).

საქმე იმაშია, რომ ჩვენ ამ კვანძში მოვაღწიეთ ჯერჯერობით მხოლოდ ირიბად, ორი დიაგონალური სვლით (14+14). სხვა გზა ალგორითმს ჯერ არ დაუნახია.



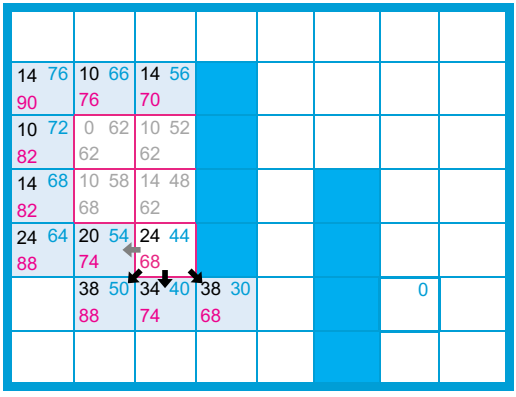
შემდეგი იტერაცია: ახლა Fcost ის მინიმალური მნიშვნელობა 68 - ა, ის ორ კვანძს აქვს (10,58) და (24,44).

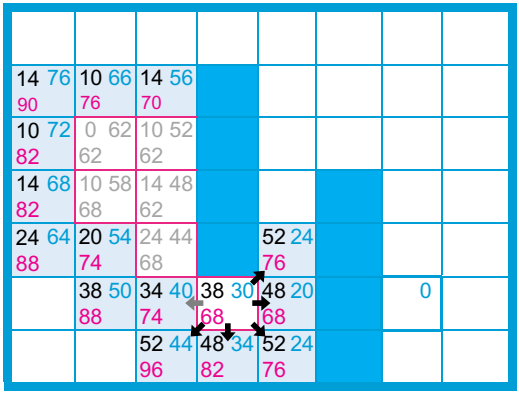
აქედან მინიმალური Gcost 10-ია, და გადავდივართ ამ კვანძზე (სურ.).

ამ კვანძის სამი მეზობელი უკვე დახურულია, ოთხი უკვე გახსნილია და ერთი - გასახსნელი.

ამ ეტაპზე საინტერესოა რა ხდება იმ კვანძში, რომელიც მიმდინარე კვანძის ქვემოდაა: წინა ნაბიჯზე მას მიენიჭა მნიშვნელობა 28, ორი დიაგონალური გადასვლის გამო. ახლა სიტუაცია გამოსწორდა და მას მიენიჭა ახალი მნიშვნელობა 10+10 = 20.

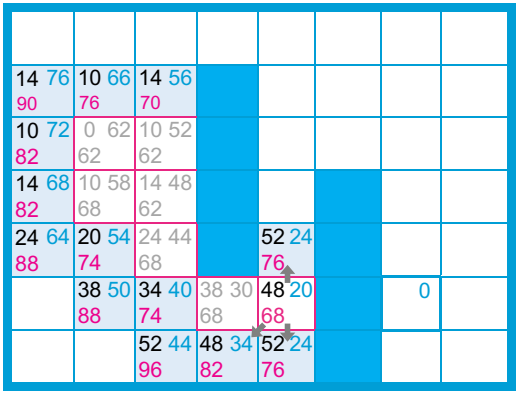
შემდეგ ეტაპზე გადავდივართ კვანძზე 68 (24, 44). (სურ ...) აქედან იხსნება სამი ახალი კვანძი, რომლითგან მინიმალური მნიშვნელობა აქვს კვანძს Fcost = 68, რომელზეც გადავდივართ შემდეგ ნაბიჯზე და იქიდან იხსნება ხუთი ახალი კვანძი (სურ. ...).

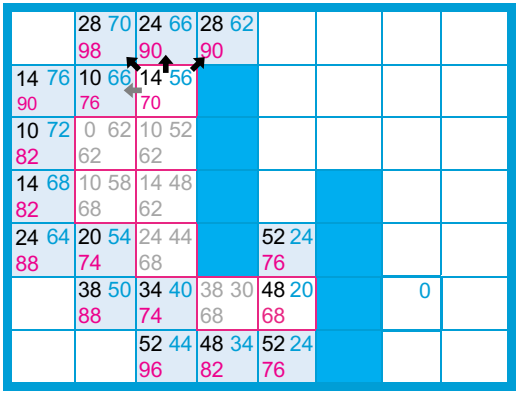




მიმდინარე კვანძი 68 (48, 20) ესაზღვრება მხოლოდ გახსნილ კვანძებს (სურ ...).

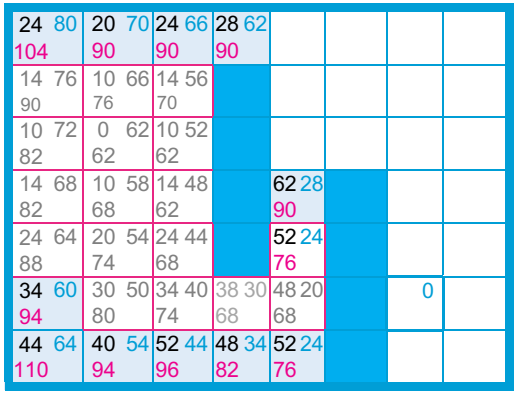
ახლა კი აღმოჩნდა, რომ ყველაზე დაბალი Fcost აქვს საწყისი კვანძის მეზობელს - კვანძს 70 (14, 56). გადავდივართ იქ და ვხსნით სამ ახალ კვანძს. (სურ ...)





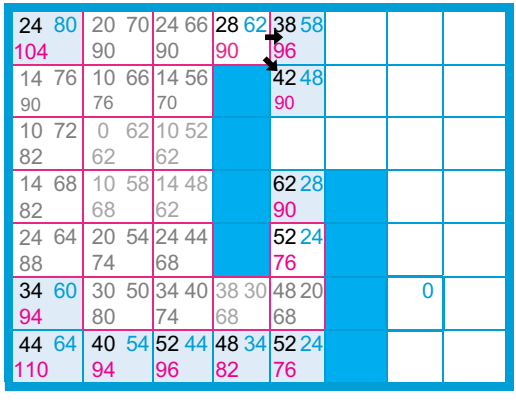
აქ გვაქვს საქმე საინტერესო ტენდენციასტან: ალგორითმმნა „იგრძნო“ მეორე კედლის არსებობა და ის, რომ აქამდე გაყვანილი გზა გაგრძელდა და ამ ფონზე კვანძი 70 (14, 56) საინტერესოდ გამოიყურება.

ამასობაში ღია კვანძებში მინიმალური მნიშვნელობა აქვთ კვანძებს მნიშვნელობით 74 - ამ კვანძს შორის ისევ ვირჩევთ იმას, რომელსაც ნაკლები Gcost: 74 (20, 54) (ჩვენ კი ვიცით, რომ ეს არასწორი მიმართულებაა, მაგრამ ალგორითმი ამას გაივლის) შემდეგ გადავდივართ მეორე კვანძზე: 74 (34, 40)



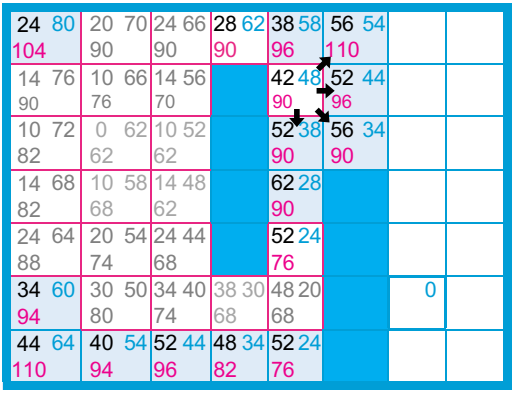
შემდეგ—კვანძები 76 (10,66) და 80 (30, 50) , 82, 82, და 88 (სურ ...).

დადგა რიგი კვანძებისთვის მნიშვნელობით 90. გავივლით პირველ ორ კვანძს Gcost - ებით 20 და 24, რაც არაფერს შეცვლის, მაგრამ შემდეგ გადავალთ კვანძზე 90 (28,62). ეს კვანძი გახსნის ორ ახალს - 96 (38, 58) და 90 (42, 48).

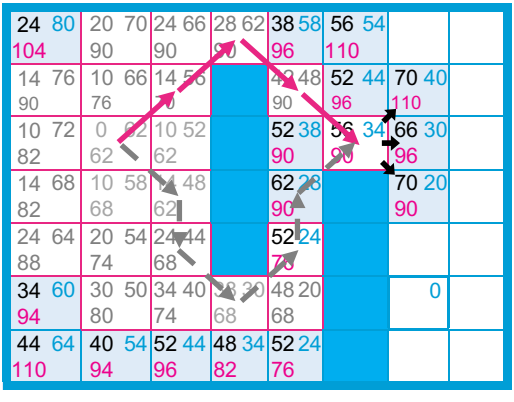


ახლა ერთმანეთის პირისპირ განლაგებულია ორი კვანძი თანაბარი Fcost (90) - ით, მაგრამ სხვადასხვა Gcost (42 და 62). ახლა გამოჩნდა, თანაბარი Fcost-ის შემთხვევაში თავიდან რატომ ვხსნით კვანძს უფრო დაბალი Gcost-ით.

ამ ორი, თანაბარი Fcost მნიშვნელობის (90) კვანძს შორის არის ორი კვანძი (Hcost მნიშვნელობებით 38 და 34).



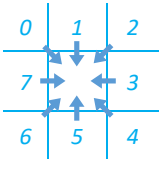
ამ კვანძებზე გადასვლა უნდა მოხდეს იმ კვანძიდან, რომელმაც საწყისი კვანძიდან ნაკლები გზა გაიარა, რაც ასახულია Gcost მნიშვნელობაში (42 ზედა კვანძში, 62 ქვედა კვანძში, განსხვავება 20, ანუ ზედა გზა ორი ორთოგონალური გადასვლით მოკლეა) - ორივე გზა ნაჩვენებია სურ (...)



კვანძ 90 (56, 34) - ში გადასვლის მერე ალგორითმი მარტივად, სამ სვლაში, ჩადის სამიზნე კვანძში. სამიზნე კვანძში ჩაღწევის შემდეგ იქ გამოთვლილი Gcost (ის იქნება 90-ს ტოლი) გვაძლევს ზოგად ინფორმაციას გავლილი გზის სიგრძეზე. ჩვენ შემთხვევაში ეს იყო 5*14 + 2*10. სიგრძე გამოვთვალეთ, მაგრამ როგორ დავადგინოთ თვითონ გზა?

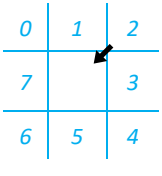
ამისთვის, კვანძის Fcost მნიშვნელობის შეცვლისას უნდა აღვნიშნოთ, თუ საიდან, რომელი კვანძიდან მივაღწიეთ მას.

სიმარტივისთვის შეიძლება მხოლოდ მიმართულების შენახვა: საიდან, რომელი მიმართულებიდან ესაზღვრება ამ კვანძს მისი წინამორბედი. მაგალითად, კვანძის მონაცემთა სტრუქტურაში იქნება მთელი ცვლადი prev, რომლის მნიშვნელობა იქნება მთელი რიცხვი 0-დან 7-მდე.



ასევე გამოცხადებულია გადაადგილების მასივები:

```
PRVx[] {-1,  0,  1, 1, 1, 0, -1, -1 } და
PRVy[] {-1, -1, -1, 0, 1, 1,  1,  0 }
```



ასე, კვანძში `A[i][j]` -ში ცვლადი prev თუ იქნება 2-ის ტოლი, მაშინ ამის წინამორბედი კვანძი იქნება `A[i+1][j-1]`

ახლა კი ჩამოვწეროთ ალგორითმის მსვლელობა:

1. ვქმნით რიგს პრიორიტეტით - იმ წერტილებისთვის, რაც გაიხსნება, მაგრამ ჯერ არ იქნება

2. დახურული (ჩვენ სქემებში - ლურჯო ფონის მქონე ველები). მთავარი პრიორიტეტი იქნება ამ კვანძების Fcost მნიშვნელობა, მეორე - Gcost. ამ წერტილებზე გადის ალგორითმის მიმდინარე ანალიზის „ფრონტი“. დავარქვათ ამ რიგს „ფრონტის ხაზი“.

2. საწყისი კვანძი იწერება რიგში.

3. ციკლი:

- თუ რიგი ცარიელია, ვასრულებთ პროგრამას.
- ვიღებთ ფრონტის ხაზის თავში მყოფ წერტილს:
- ვანიჭებთ ამ წერტილს დახურული კვანძის მარკერს
- ამ წერტილის ყველა მეზობლისთვის, ვინც არ არის კედელი ან არ აქვს უკვე დახურული კვანძის მარკერი:
- თუ მეზობელი - მიზანია, გავდივართ ციკლიდან და ვიწყებთ უკანა გზის ათვლას
- თუ არა: ვანგარიშობთ მეზობლის `Gcost, Hcost` და `Fcost` -ს და ვამატებთ ამ მეზობელს „ფრონტის ხაზის“ რიგში.

// ციკლის დასასრული

4. უკანა გზის ათვლის ბლოკი:

ვიწყებთ სამიზნე კვანძით
ციკლი, სანამ არ მოვხვდით საწყის კვანძში:
გადავდივართ წინამორბედ კვანძში